

P?S/8 Binary File Internal Description

Last edit: 27-May-2019 cjl

P?S/8 supports binary files in a variety of contexts, primarily as input to the multi-purpose binary utility program known as *P?S/8 BIN*.

Note: P?S/8 BIN also supports binary paper-tape conversion and other options. Documentation of P?S/8 BIN is available as a separate document.

The binary file format is known as *Slurp* binary; this term was coined by its inventor, the legendary Richard Lary, generally known as the best PDP-8 programmer in the history of mankind. The term is applied as an attempt to explain how the loader works without requiring a separate buffer (as is usually the case), even when loading from certain storage devices traditionally associated with block-oriented access. In P?S/8, each system device generally has a companion Slurp loader component, which is used by the P?S/8 BIN utility to load binary files on any particular system. This is a customized routine designed for the same device as the system device handler normally resident in the system kernel area, but used as a temporary replacement routine for the express purpose of efficiently loading binary files.

Note: The P?S/8 BIN program should not be confused with the name usually associated with DEC standard binary paper-tapes, which are generally loaded by the DEC *BIN* paper-tape loader. That said, P?S/8 BIN does support certain operating modes that handle DEC BIN (and DEC *RIM*) binary paper-tapes, including the conversion of DEC BIN format paper-tapes from/to P?S/8 Slurp-format binary files. When the terms file and BIN are used in the same sentence, this invariably refers to Slurp format binary files and the P?S/8 BIN program that loads the files.

The advantages of a hardware-specific custom binary file loader include:

- 1) No reserved memory areas beyond the prevailing system convention (which is generally locations 07600-07777 unless a device handler extension is required). Where relevant, the system device handler extension occupies locations x6000-x7777 where x is the highest available field of extended memory.
- 2) The ability to load memory with the system device write-protected (where applicable).
- 3) Surprisingly rapid program loading; this is often contrary to the expectations of programmers taught to believe what the Slurp loader actually accomplishes must be "impossible".

The best example of a Slurp binary loader implementation is the one customized for the TC01/TC08 DECTape controller.

The TC01/TC08 system device handler loads into 07600-07777 and provides (among other resources) a read/write subroutine call to access the various logical drive units (which are implemented as either TU55 or TU56 DECTape drives). Certain memory locations are reserved for various system parameters and passed file lists used by various system programs. The operating system can always be restarted at location 07600 (which is the universal starting address for all single-user PDP-8 operating systems).

The P?S/8 BIN program is typically used to replace the contents of memory locations 07600-07745 with the Slurp binary loader; this includes a stand-alone bootstrap (starting at 07600) which can be used to reload the operating system after the program has been loaded and executed, and no longer needed in memory.

While the Slurp loader is in memory, no block-oriented read/write services are available. Memory locations 07746-07777 are preserved to pass the starting block and logical unit information for each Slurp binary file to be loaded, as well as the overall program starting field and address information.

The Slurp loader performs search operations on the DECTape-based logical drive units using the passed starting block and logical unit file parameters as were originally passed to the P?S/8 BIN program (which loaded the Slurp loader over the system device handler). When the proper starting block is located, the program is then directly loaded into memory using the novel technique of a single-word input buffer within the Slurp loader code page.

The reason this is possible is that 12-bit words are brought into memory by the three-cycle data break mechanism; it is possible to detect individual word transfers by monitoring the associated word count (and/or current address) locations for an expected change. Once the word count changes, it is reset for the next transfer and the latest word transferred can then be used to effect the latest binary operation (which is primarily to load the latest word into memory and increment the location counter).

Note: By nature, most PDP-8 binary programs consist of contiguous words; this is properly exploited in both the Slurp binary file format and the DEC BIN paper-tape format; however, DEC BIN format is much more inefficient and based on an 8-bit medium that requires unpacking into 12-bit words (and includes the overhead of leader/trailer frames).

The Slurp loader internal operations are analogous to those of the paper-tape binary loaders which use a completely different data format on paper-tape media. All needed operations are provided in either loader; however, paper-tape loading is far slower and designed for that slow medium. Slurp loading is nearly as efficient as block image loading, yet is random access loading much like paper-tape loading.

In the case of the TC01/TC08-specific Slurp loader, it is necessary for each word transfer to be completed nominally in 133 microseconds; this is easily achieved since the loader programming is short and efficient.

The loading process continues while the DECTape read operation spans multiple blocks; tape motion is continuously forward without turn-around. Once the Slurp end-of-file indication is processed, the next passed file parameters are processed, generally requiring a new block search. Eventually all of the passed file parameters are processed and the loading process is complete. At this point, a starting field and starting address are used to start the program as indicated in the original execution-class command. (This information was provided by initialization code in the P?S/8 BIN program as determined by command-line parameters provided by the user. Additional command-line options may have been processed prior to moving the Slurp loader through memory over the system device handler.)

All P?S/8 releases support an alternative generic Slurp binary loader that uses virtual techniques to load binary files; however, due to the virtualization overhead, the loading process will tend to take additional time to complete. Additionally, write enable is required on the logical device unit where the P?S/8 BIN program resides as the P?S/8 BIN program includes its own virtual buffer area.

Note: In the next full release of P?S/8, this implementation will likely be slightly modified to address reserved scratch blocks on any logical device unit; as a result, the overall size of the P?S/8 BIN program will be somewhat shorter. Future releases may use the same reserved blocks for alternate purposes to be determined later.

While the majority of P?S/8 configurations include a Slurp loader customized to the same hardware as the system device handler, a few system configurations force the use of the virtual Slurp binary loader:

- 1) The RF08 is the only DEC three-cycle data break (DMA) device that cannot support a slurp loader; this is due to the high data rate which doesn't allow sufficient time between word transfers to handle all logical sequences of the Slurp loader.
- 2) The RK8E and several other single-cycle data break devices. No method is provided to detect word transfers; the flag raises only at the end of block transfers.

In general, all other supported devices transfer data by asynchronous read operations; this allows a device-specific Slurp loader implementation without timing considerations.

An advantage of the virtual Slurp loader is that the system device handler is still resident after the program is loaded; however, all customized Slurp loaders support optional reloading of the system device handler (a process which admittedly appears at first glance to be both counter-intuitive and impossible). The only restriction on this usage (for the purpose of reloading the system device handler) is lowering the normal maximum input file count of 17 to 16 total input files; this is indicated by invoking the BIN command with the `/I` command-line option switch. As a practical matter, the lowered limit exceeds most binary loading scenarios as Slurp binary files contain nearly 3/7 of a memory field each; the restriction is thus largely moot.

While there are many additional details regarding loading options, the reader is directed to documentation of the P?S/8 BIN program (available separately) for more information; the primary purpose of this document is to describe details of the Slurp file format.

Slurp format can be broken down into a series of contiguous file blocks, each independent of other blocks with the exception of the prevailing loading address and memory field in effect carried forth from the previously loaded data block.

Note: If a single project spans more than a single binary file, all files need to be loaded in the order originally created.

Each block of Slurp format binary consists of a series of seven-word groups in contiguous order. Except for the end-of-file block, there are always 18 seven-word groups in any block; each block also contains two waste words (since the blocks are logically 128 words each).

Note: DECTape blocks are actually 129 12-bit words each; however, the 129th word is ignored throughout P?S/8. On the TC01/TC08 system, the word count is generally set to -128 as required; as such, the 129th word is never transferred. The majority of PDP-8 operating systems are designed to operate in a like manner.

Each seven-word group breaks down as follows:

- 1) A control word consisting of six bit pairs used to control the disposition of the data words that immediately follow.
- 2) The first of six data words; the action taken regarding this word is determined by bits[0-1] of the control word.
- 3) The second of six data words; the action taken regarding this word is determined by bits[2-3] of the control word.
- 4) The third of six data words; the action taken regarding this word is determined by bits[4-5] of the control word.
- 5) The fourth of six data words; the action taken regarding this word is determined by bits[6-7] of the control word.
- 6) The fifth of six data words; the action taken regarding this word is determined by bits[8-9] of the control word.
- 7) The sixth of six data words; the action taken regarding this word is determined by bits[10-11] of the control word.

Each pair of control word bits breaks down as follows:

- 1) 00 - The corresponding word is a data word; load this word into the memory location pointed to by the internal load pointer then increment the load pointer for the next data operation. The loading field value in effect determines the memory field used for data loading.
- 2) 01 - The end-of-file has been reached. The Slurp loader has finished processing this file; loading continues with the contents of the next passed file (if any).
- 3) 10 - The corresponding word is an origin setting. set the load pointer to the contents of this word.
- 4) 11 - The corresponding word is a field setting of the form *CDF nn* where *nn* is the new loading field in bits[6-8]. This word will be stored inline (within the Slurp loader) over an existing CDF instruction; this instruction will be executed before data words are stored; as such, all data words (if any) will be loaded into field *nn*.

Despite the fact that Slurp format is inherently random access (and not block-oriented), the efficiency of data loading approaches 6/7 of that of core-image loading.

Note: P?S/8 system programs are stored in core-image format for the fastest possible loading speed; however, a binary program file must first be saved in core-image format before the system device handler can be used to load and execute the program.

Practical implementations of programs that create Slurp format binary files create predictable characteristics within the file. This allows other utilities to determine if a file might be in Slurp binary format. Additionally, multiple files may be loaded in the same overall loading operation. Specific validation safeguards must be present to prevent potential loading mishap.

A short example program follows which creates a Slurp format binary output file by program assembly with P?S/8 *PAL*.

Note: The example program output is short enough to be entirely contained in a single file block which will be illustrated below in Figure 1.

```
.FETCH EXAMPLE          This brings the source program file into the edit buffer.

.LI                      The contents of the file are listed on the system console
                        device.
100    /                EXAMPLE OF P?S/8 "SLURP" BINARY FILE FORMAT
110
120    FIELD    7
130
140    *1000
150
160    START,  CLA CLL    /CLEAN UP
170           CIF CDF 00  /GOING TO FIELD 0
180           JMP I   .+1 /GOING THERE
190           7600      /THROUGH HERE
200
210           $          /THAT'S ALL, FOLK!

.PAL EXAMPLE /B /L /T    This assembles the program producing binary output
                        into block 0020 which can be addressed as the % file.
                        Additionally, a minimal listing output is directed to
                        the system console device.
```

```

/          EXAMPLE OF P?S/8 "SLURP" BINARY FILE FORMAT

0007          FIELD    7
*1000          *1000

071000  7300  START,  CLA CLL      /CLEAN UP
071001  6203          CIF CDF 00    /GOING TO FIELD 0
071002  5603          JMP I    .+1  /GOING THERE
071003  7600          7600          /THROUGH HERE

$          /THAT'S ALL, FOLK!

```

NO ERRORS DETECTED

NO LINKS GENERATED

4K MEMORY UTILIZED

1 FILE CREATED

1 SYMBOL

This example program illustrates all features of the slurp binary format file in a single block printout.

```

BLOCK: 0:0020

0000  5340 0200 0200 6271 1000 7300 6203 0125  *+ B@B@29H@;@2CAU*
0010  5603 7600 4503 5441 7332 1125 0010 7650  *.C>@%C,!;ZIU@H>(*
0020  5212 7346 1013 3025 4452 3425 6201 5212  **J;&HKXU$*\U2A*J*
0030  7201 4503 5441 7344 1013 3025 4452 1425  *:A%C,!;$HKXU$*LU*
0040  7004 7130 3425 6201 5231 3145 1117 7640  *8D9X\U2A*YY%IO> *
0050  4437 1337 4200 1325 4200 0000 2110 2107  *$_K_"@KU"@@QHQG*
0060  1323 3730 0000 2105 2053 1335 4200 1731  *KS_X@@QEP+K]"@OY*
0070  7640 5274 1324 3733 1727 3722 1337 4200  *> *<KT_[OW_RK_"@*
0100  4726 0321 1145 1015 4726 0347 1614 0047  *'VCQI%HM'VC'NL@'*
0110  3133 3143 3144 3102 3117 1336 4200 4721  *Y[Y#Y$YBYOK^"@!Q*
0120  5732 2501 4165 2442 5277 4410 4503 2767  */ZUA!5T"?$H%CW7*
0130  4376 4126 3447 4065 3542 4421 4430 4337  *#>!V\' 5]"$Q$X#-*
0140  1314 1104 4042 4057 3042 5275 0750 7240  *KLID " /X"*=G(:-*
0150  0104 0200 0137 0200 0246 7000 0234 7711  *ADB@A_B@B&8@B\?I*
0160  0007 0200 0155 0000 3517 5247 3541 4771  *@GB@A-@@]O*'!]9*
0170  3542 4736 1363 4427 0733 7640 2442 1341  *]"'AK3$WG[> T"K!*

```

Figure 1: Dump of binary output file (block 0020)

Figure 1 above shows the contents of block 0020 just after the assembly of the example file. A description of each word is as follows:

- 0000) 5340 - This is the first control word in the block which indicates the disposition of the next six data words.
- 0001) 0200 - All binary files automatically generate an opening *200 origin setting. This is universally the case with all PDP-8 assemblers.
- 0002) 0200 - P?S/8 PAL outputs the prevailing current origin at the start of each binary output file. In this example, this is redundant to the previous word only because this is the first (and only) output file. The prevailing field setting is contained within the slurp loader; it is initialized to load into field zero by default.

Note: Future releases of P?S/8 PAL may also include the prevailing field setting before the prevailing origin setting. This will allow more robust file validity checking and the elimination of specific loading order of output files created by a particular assembly. Future releases of P?S/8 BIN will include corresponding changes when converting binary paper-tapes to P/S/8 Slurp format binary files to achieve equivalent goals.

- 0003) 6271 - The source program starts with a FIELD 7 directive; as such, a corresponding CDF 70 (6271) instruction is placed in the output file.

Note: Most PDP-8 assemblers can place an automatic *200 after FIELD directives. This is generally not necessary as good programming practice demands the use of an explicit new origin setting consistent with program design. *OS/8 PAL8* and P?S/8 PAL include command-line option switches to enable this feature; by default this action is disabled to minimize the binary file size. Binary output intended to be punched to binary paper-tapes and ultimately loaded by the DEC "self-starting" binary loader must enable this feature to avoid a serious loading error, a problem which was identified, but never remedied.

- 0004) 1000 - The source program includes *1000 at this point.

- 0005) 7300 - The source program includes a CLA CLL instruction.

- 0006) 6203 - The source program includes a CIF CDF 00 instruction.

- 0007) 0125 - This is the second (and final) control word in the block (and entire program) which indicates the disposition of the next six data words. (Actually, this is partially moot as the end-of-file is indicated before the end of this control word; the control word is often fleshed out with additional end-of-file control bits.)

- 0010) 5603 - The program includes a JMP I 0203 instruction.

- 0011) 7600 - The program includes the address value of 7600 used to exit the program.

- 0012) xxxx - The end-of-file point has been reached. words 0012-0175 are ignored. Additionally, words 0176-0177 are ignored in all Slurp binary file blocks.

[end-of-file]